OBJECT-ORIENTED ANALYSIS & DESIGN · WEEK 3

Class–Object Models & CRC Cards

Design the classes, the objects will follow.



This Card Made a Whole Team Miserable

The story

Week 1 of an e-commerce project, a confident team writes this single card for their ordering system.

Week 6: every bug fix touches it. Every new feature touches it. Nobody dares refactor it.

Your job (3 minutes): discuss in teams, what is wrong with this card? Be specific.

OrderManager

RESPONSIBILITIES

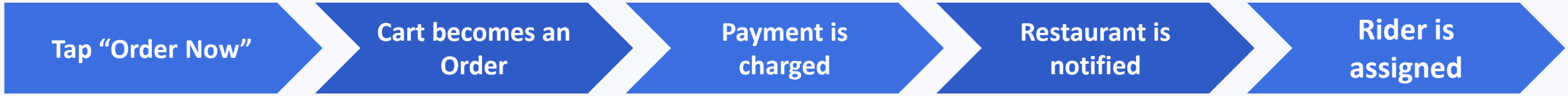
- Place order
- Calculate total price
- Check inventory
- Charge payment
- Generate invoice PDF
- Send email confirmation
- Process refunds
- Track order status

COLLABORATORS

- Customer
- Inventory
- PaymentGateway
- EmailService
- PDFGenerator
- RefundService

Let's revisit this slide, and you'll redo this process with proper vocabulary, cohesion, and coupling, and rebuild the card set yourself.

You Tap “Order Now”. What Moves Inside?



Real things

Restaurant · Rider · Food bag, you could point at them in the street.

Code-only things

Cart · Order · PaymentGateway, they exist only as objects in memory.

The modeling question

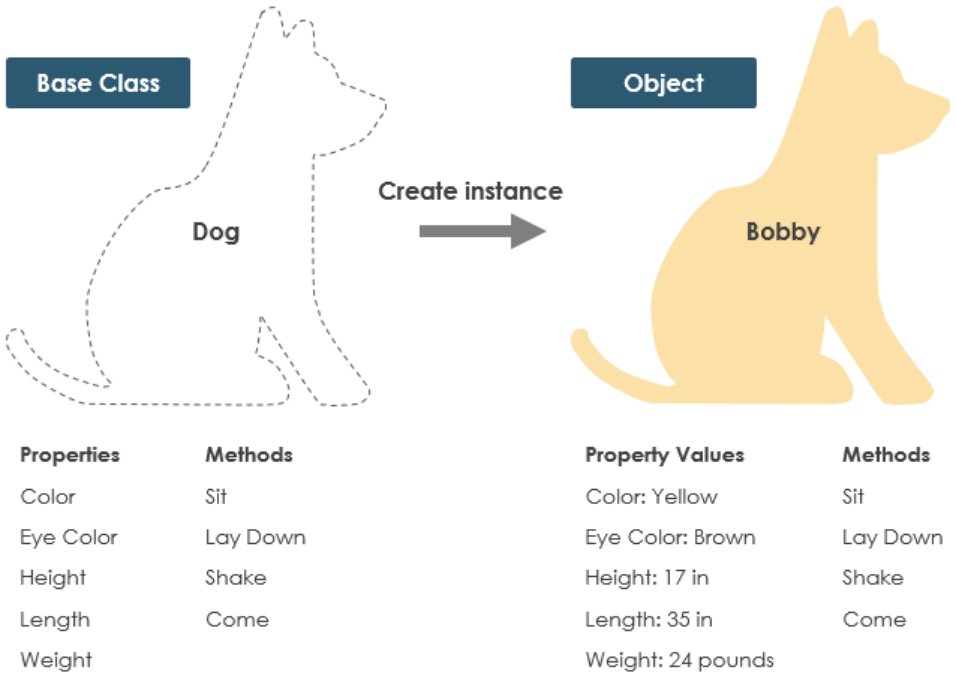
How do we name, organize, and connect these “things”: real or invented?

◆ **Two words carry all the weight in OO:**

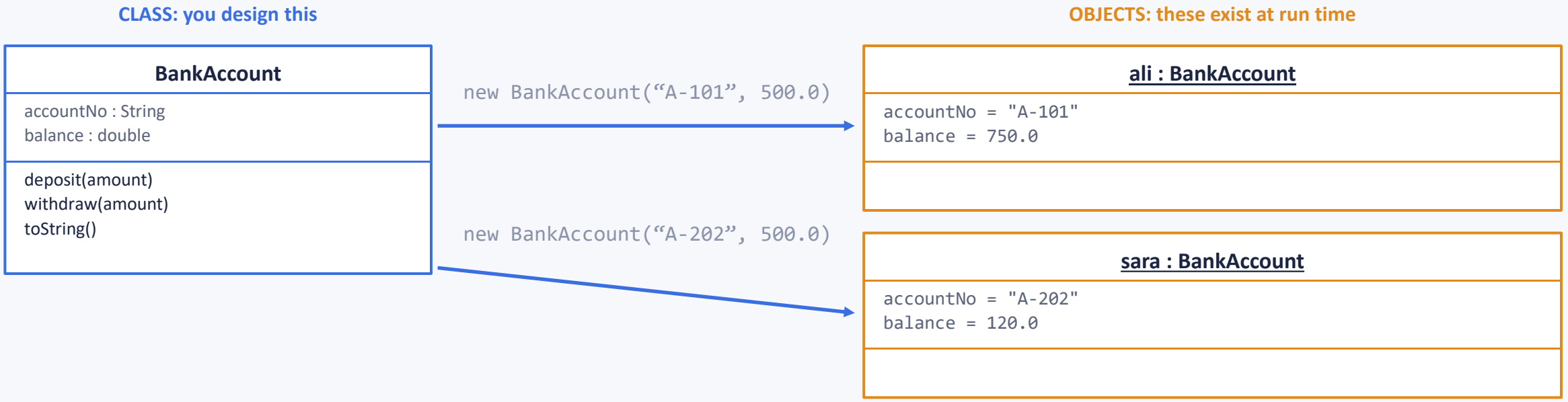
CLASS: the idea, the blueprint, what you design. **OBJECT**: the live thing in memory, what actually runs.

Class Revisited

- A blueprint for an object
- The entire point of Object-Oriented Design is not about objects; it's about classes as we use the classes to create objects.



Class = Blueprint. Object = Instance.



Class level: `Student enrolls in→ Course` **Object level (association link):** `ali:Student enrolled in→ cs1046:Course`

Underline = instance. `ali:Student` is one object of class Student, same idea at two zoom levels.

◆ **Why we say "OOD is about classes":**

design conversations happen at the class level; running programs are populated with objects. You cannot design "an object", you design the class, and objects follow.

Every Object Has Four Things Going For It

① TYPE

Which class it was born from, decides what it can do.

ali is a `BankAccount` → it can deposit(), withdraw()

② IDENTITY

What makes it THAT object, regardless of its data.

accountNo `A-101`, two identical-looking accounts are never the same account

③ STATE

Its attribute values right now, state changes over time.

balance = `750.00` status = `ACTIVE`

④ BEHAVIOR

The operations it can perform, its methods.

deposit() `withdraw()` toString()

◆ Same class ⇒ same **type** & **behavior**. Different objects ⇒ different **identity** & **state**.

Class: Light Bulb

- Identity
  - Product number/Batch Number
- State
  - Lit or not (on or off)
- Behavior
  - Turn on
  - Turn off
  - Check whether lit



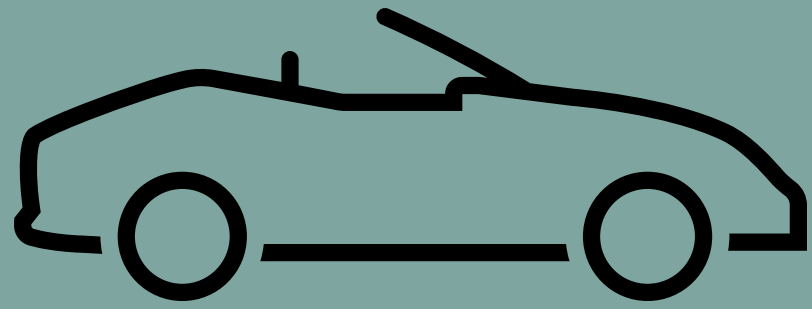
Class: Bank Account

- Identity
  - Account Number/IBAN
- State
  - Balance
- Behavior
  - Deposit
  - Withdraw
  - Inquire Balance



Class: Car

- Identity
  - Registration Number / VIN
- State
  - Distance Travelled
  - Gas Left
- Behavior
  - Drive
  - Load gas
  - Check gas level
  - Check Odometer



One Pattern, Many Domains

CLASS	IDENTITY	STATE	BEHAVIOR
`LightBulb`	serial / batch number	`on | off`	`turnOn()` · `turnOff()` · `isLit()`
`BankAccount`	account number / IBAN	`balance` · `status`	`deposit()` · `withdraw()` · `getBalance()`
`Car`	VIN / registration number	`distance travelled` · `gas left`	`drive()` · `refuel()` · `checkFuel()` · `checkOdometer()`

! **Identity ≠ State:**
two bulbs can both be OFF, they are still two different bulbs. State changes every second; identity never does. Lose this distinction and databases, undo buttons, and equality checks all break.

The Blueprint, in Two Languages

JAVA

```java
public class BankAccount {        // the blueprint
    private String accountNo;     // identity
    private double balance;       // state

    public BankAccount(String no, double bal) {
        accountNo = no; balance = bal;   // birth of an object
    }
    public void deposit(double amt) { balance += amt; }

    public boolean withdraw(double amt) {
        if (amt > balance) return false;
        balance -= amt; return true;     // guards state
    }
}
```

PYTHON

```python
class BankAccount:                    # the blueprint
    def __init__(self, account_no, balance):
        self.account_no = account_no   # identity
        self.balance = balance         # state

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount > self.balance:
            return False
        self.balance -= amount
        return True                    # guards state
```

attributes → state

methods → behavior

constructor → an object is born

> ◆ **Notice:**
> same blueprint, two dialects.

Identifying Classes (Noun Technique)

—

- Look for nouns in the problem statement
  - Filter out attributes
  - Example: 'A library member borrows books.'
  - Candidates: Library, Member, Book
  - Attribute (not class): name of member

Apply the noun-verb technique

The noun-verb technique is a simple and effective way to extract classes and attributes from a textual description of the system or problem. It involves identifying the nouns and verbs in the text and mapping them to classes and attributes respectively. Nouns usually correspond to classes, while verbs usually correspond to attributes or methods of the classes. For example, in the sentence "A librarian can check out a book to a borrower", the nouns are librarian, book, and borrower, and the verb is check out. These can be mapped to classes Librarian, Book, and Borrower, and an attribute or method checkOut of the class Librarian.

Finding Classes: Start With the Nouns

"A library member borrows books."

nouns → class candidates verbs → operations (…or a class?)

CANDIDATES

Member ✓ | Book ✓ | Library ✓ (context) | borrow → operation | "member's name" ✗ attribute

The three-question filter

? Does it have its own state worth tracking?

? Does it have its own behavior?

? Will we need many instances of it?

Fail any question → probably an attribute or an operation.

…and the reverse move: promote a verb

"borrow" has data attached: **who, which book, when due, renewed?**

When a verb needs memory, promote it to a class: **Loan** (knows due date; renews; marks returned).

! **Caveat:**
the noun technique generates candidates, not verdicts (Larman, Applying UML and Patterns). Some nouns die as attributes; some verbs get promoted. Judgment decides.

Grammar → Design: A Mapping That (Mostly) Works

The **player** **deals** a **card** from the **deck** when it is **her turn**.

player/card/deck → classes · deals → method · her turn → whose? state!

Common noun	**Class**	`Deck, Card, Player`
Proper noun	**Object (instance)**	`bobHand : Hand  ("Bob's poker hand")`
Adjective	**Attribute**	`card.suit = SPADES  ("this card is a spade")`
Action verb	**Method**	`dealer.deal(card)`
"Has" verb	**Composition / aggregation**	`deck holds cards  ("a deck has cards")`
"Is-a" verb	**Inheritance**	`SavingsAccount is a BankAccount  ✓`

✗ **Corrections corner — "be verb → inheritance":**

the classic classroom example says "a hand is a deck" → inheritance. Wrong. A hand is NOT a subtype of a deck; you can't substitute one for the other. We replaced it with SavingsAccount is a BankAccount. The is-a test must check substitutability, not grammar.

Grammar Is a Flashlight, Not a GPS

TRAP 1 · Location lies

"The card is in the deck."

An **is** of location, not inheritance.

Model it as an association/composition: **`Deck` ◆—→ `Card`**

TRAP 2 · Math lies

"A square is a rectangle."

True in geometry, dangerous in code: a `Square` subclass that inherits `setWidth()` must also change height, breaking Rectangle's contract.

The famous Liskov substitution trap. Design by substitutability, not by sentence.

TRAP 3 · Nouns lie

"The member's name changed."

`name` is a noun, but it is an **attribute of Member**, not a class.

Apply the filter: no behavior of its own, no instances to track.

◆ **The rule:**

use grammar to GENERATE candidates; use judgment (state? behavior? many instances? substitutability?) to DECIDE.

Not All Classes Do the Same Kind of Work

▣ DATA MANAGER

Stores, maintains, and provides access to data. It holds the state.

Member · TaskManager · Book

⇄ SOURCE / SINK

Generates or consumes data at the system's edges: I/O, files, external services.

PaymentProcessor · StorageHandler

◉ VIEW / OBSERVER

Presents data and reacts to changes. Displays and monitors, doesn't own the data.

Dashboard · ContactListPanel

✚ FACILITATOR / HELPER

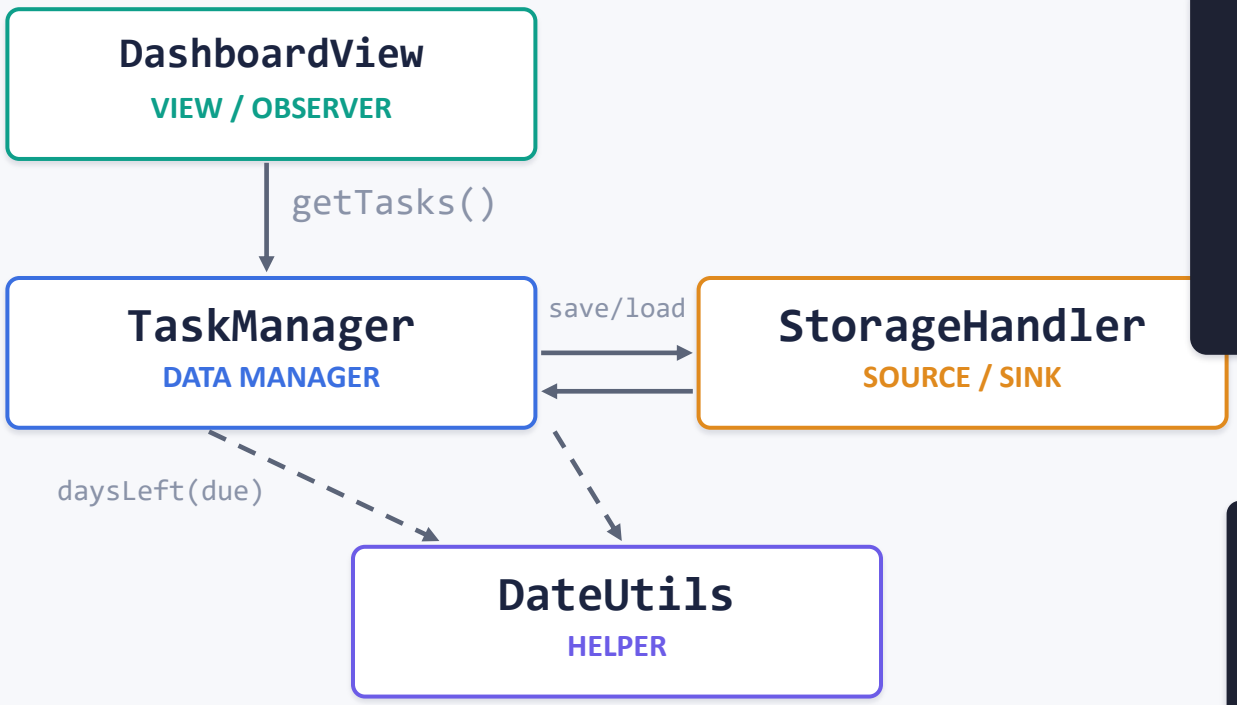
Supporting computations. Usually stateless, everyone reuses it.

DateUtils · EmailValidator

Why roles matter: they keep single responsibility alive · they tell you which way dependencies should point (views depend on data managers, never the reverse) · and "this is a view class" instantly sets expectations in a team.

Role taxonomy: Timothy Budd, An Introduction to Object-Oriented Programming (as in the base deck).

One Tiny To-Do App, All Four Roles



JAVA

```java
// HELPER — stateless utility
public final class DateUtils {
    public static long daysLeft(LocalDate due) {
        return ChronoUnit.DAYS.between(
                    LocalDate.now(), due);
    }
}
```

PYTHON

```python
# HELPER — plain functions, no state
from datetime import date

def days_left(due: date) -> int:
    return (due - date.today()).days
```

◆ **Look at the arrow directions:**

the arrows only point one way, the view depends on the data manager; TaskManager has no idea a UI exists. Flip that arrow and every UI tweak breaks your business logic.

Objects You Can Touch, and Objects Only Code Has

REAL-WORLD (DOMAIN) OBJECTS

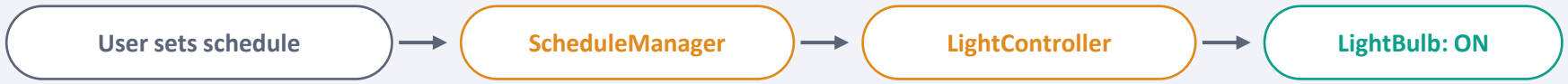
The nouns the problem statement hands you:

- Driver
- Passenger
- Car
- LightBulb
- Thermostat
- DoorSensor

SYSTEM (SOFTWARE-ONLY) OBJECTS

The coordinators the requirements demand:

- TripManager
- PaymentGateway
- LightController
- ScheduleManager
- SensorDataLogger



meanwhile: `Thermostat → SensorDataLogger records temperature over time`

◆ The insight:

the domain gives you nouns; the requirements give you managers. Neither is more “real” in the model, both are first-class objects.

CRC Cards: A 1989 Idea That Refuses to Die

Late 1980s

Ward Cunningham invents CRC cards to document collaborative design decisions, the first version is a HyperCard stack.

OOPSLA 1989

Cunningham & Kent Beck present “A Laboratory for Teaching Object-Oriented Thinking”: index cards teach OO design to people who don’t program.

1989–1990

Rebecca Wirfs-Brock and colleagues formalize Responsibility-Driven Design (OOPSLA ’89 paper; book Designing Object-Oriented Software, 1990).

Today

A staple of XP and agile design workshops, physical cards, table-top rearranging, role-play walkthroughs; often followed by sequence diagrams.

> *CRC cards avoid a great deal of drawing and erasing.*
>
> — Martin Fowler, UML Distilled

Why index cards?

- Small card = less room for complexity
- Portable = rearrange on a table mid-argument
- Cheap = cheap to throw away and redo

◆ **CRC = Class · Responsibilities · Collaborators:**
a tool old enough to vote, still the fastest way to argue about responsibilities before code locks them in.

Anatomy of a CRC Card

RESPONSIBILITIES

Short verb phrases: what the class DOES and what it KNOWS.

Wirfs-Brock's two kinds: **knowing** (know title, know availability) and **doing** (provide details, mark borrowed).

CLASS: the name on the card

Book

RESPONSIBILITIES	COLLABORATORS
– know title, ISBN & copy count – track availability – provide details to the catalog – mark copies borrowed / returned	– Member – LibraryCatalog

COLLABORATORS

The classes this card must talk to in order to do its job.

A collaborator is never 'just there', each one serves at least one responsibility.

Reading the Book card: to "provide details to the catalog", Book needs LibraryCatalog. To "mark borrowed", it needs to know which Member. **Every collaborator earns its place.**

Three Rules That Keep Cards Honest

RULE 1

Short verb phrases

“Track availability” ✓, three words, starts with a verb.

If a responsibility wraps to a paragraph, it’s not a responsibility, it’s a project.

RULE 2

WHAT, not HOW

✓ “Validate payment” ✗ “Encrypt card via AES-256 and POST to bank API”

HOW-free cards keep options open; encryption is an implementation decision for later.

RULE 3

Small enough to stay honest

If responsibilities overflow the left half, or collaborators overflow the right, that’s a smell, not a stationery problem.

Heuristic, not law: the card is a pressure gauge for cohesion & coupling.

Responsibility ✓

VERB PHRASES · DOMAIN LANGUAGE

- Validate payment
- Track order status

COLLABORATORS

- none needed

Responsibility ✗

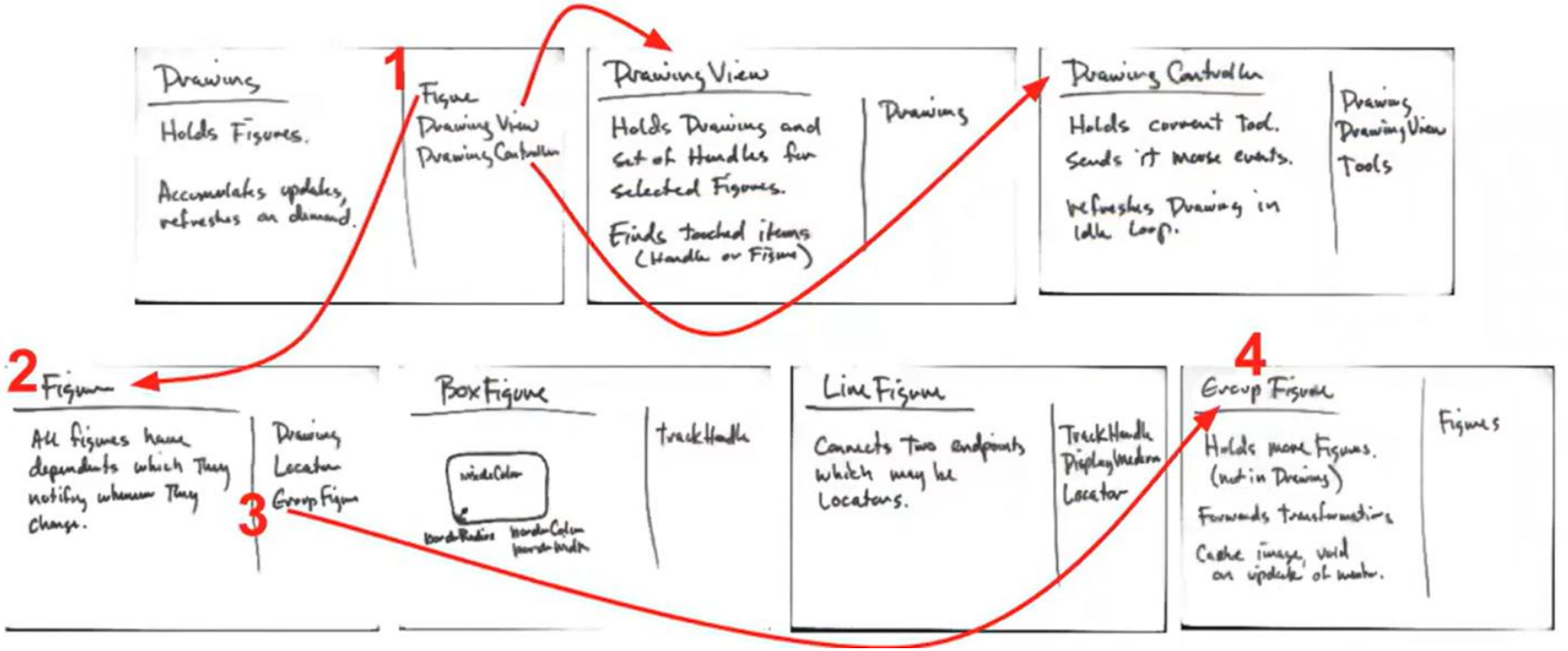
IMPLEMENTATION LEAKED IN

- Encrypt card using AES-256 and send to bank API

COLLABORATORS

- none needed

The First CRC Cards



CRC Design Guidelines

- Too many responsibilities → low cohesion
- Too many collaborators → high coupling
- Responsibilities should be short verb phrases

Cohesion: Does This Class's Stuff Belong Together?

BEFORE: one card, mixed concerns

KitchenAppliance

RESPONSIBILITIES	COLLABORATORS
– turn on / off	– Outlet
– make toast	– Bread?
– brew coffee	– Beans?
– play music	– Spotify?

LOW COHESION

split!

AFTER: one purpose per card

Toaster

- know browning level
- make toast

CoffeeMaker

- know water & beans
- brew coffee

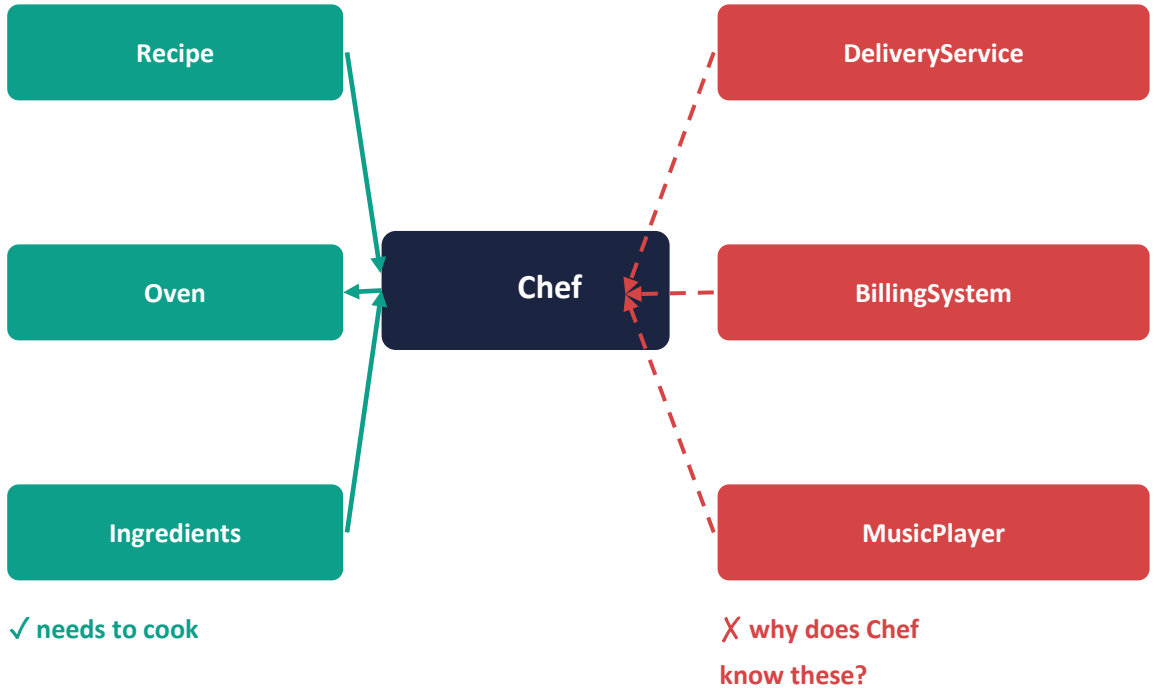
MusicPlayer

- know playlist
- play music

shared on/off? → an `Appliance` base class (that's inheritance doing its job).

◆ **Cohesion = how closely the responsibilities INSIDE one class relate to each other.** "Play music" shares nothing with "make toast". High cohesion ⇒ one focused purpose (the Single Responsibility Principle in card form).

Coupling: How Many Other Classes Does It Drag Along?



THE FIX, give each job an owner

`DeliveryService`
used by a DeliveryController, not by Chef

`BillingSystem`
used by an OrderManager, one place knows billing

`MusicPlayer`
used by whoever manages ambience, certainly not Chef

Result: a change in billing ripples through **one** class, not through every class that ever cooked an egg.

◆ **Coupling = how strongly classes depend on each other.** Low coupling ⇒ changes stay local. Cohesion looks inside the box; coupling looks between boxes, two readings of the same card.

Rehab for a God Class: Order

BEFORE

Order

RESPONSIBILITIES	COLLABORATORS
– computeOrderTotal – updateInventory – validatePayment – formatInvoicePDF – sendConfirmationEmail	– everything…

split by concern →

AFTER: one concern per card

Order

RESPONSIBILITIES	COLLABORATORS
– store items – compute total – track status	– Inventory – PaymentProc

PaymentProcessor

RESPONSIBILITIES	COLLABORATORS
– validate & charge payment – process refunds	– Customer – Order

InvoiceService

RESPONSIBILITIES	COLLABORATORS
– create invoice – generate PDF	– Order

EmailService

RESPONSIBILITIES	COLLABORATORS
– send confirmation – send updates	– Customer – Order

+ InventoryService gets its own card (track stock, reserve/release).

Each color = a different concern.

5 concerns in 1 card = 5 reasons to change.

✓ **Why this is better:**

each card now does one thing, its name sets expectations, and a change (say, a new tax rule) lands on exactly one card.

From Card to Code: the Whole Point

PaymentProcessor

RESPONSIBILITIES	COLLABORATORS
– validate & charge payment – process refunds	– Customer – Order

CLASS → class name

RESPONSIBILITY → method

COLLABORATOR → field / parameter you talk to

JAVA

```java
public class PaymentProcessor {     // CLASS
    private final Customer customer; // COLLAB
    private final Order order;       // COLLAB

    public boolean charge() {        // RESP. 1
        double amt = order.getTotal();
        if (order.isPaid()) return false;
        if (!customer.hasFunds(amt))
            return false;            // validate
        customer.debit(amt);         // charge
        order.markPaid();
        return true;
    }
    public void refund() {           // RESP. 2
        if (!order.isPaid()) return;
        customer.credit(order.getTotal());
        order.markRefunded();
    }
}
```

PYTHON

```python
class PaymentProcessor:          # CLASS
    def __init__(self, customer, order):
        self.customer = customer    # COLLAB
        self.order = order          # COLLAB

    def charge(self) -> bool:       # RESP. 1
        amt = self.order.total
        if self.order.is_paid: return False
        if not self.customer.has_funds(amt):
            return False            # validate
        self.customer.debit(amt)    # charge
        self.order.mark_paid()
        return True

    def refund(self) -> None:       # RESP. 2
        if not self.order.is_paid: return
        self.customer.credit(self.order.total)
        self.order.mark_refunded()
```

Runnable (with Customer & Order stubs + demo output): `code/PaymentProcessor.java` · `code/payment_processor.py`

Iterative Refinement

- Using CRC cards early and lightly, then revising: moving responsibilities, splitting classes, adjusting collaborators.
- The card method is low cost and meant to be changed oft
- Role-playing scenarios helps find missing collaborations or

Class: OrderManager	
Responsibilities - Place order - Calculate total price - Check inventory - Charge payment - Generate invoice PDF - Send email confirmation - Process refunds - Track order status	Collaborators - Customer - Inventory - Payment Gateway - Email Service - PDF Generator - Refund Service

Class: Order	
Responsibilities - Store items - Track order status	Collaborators - Customer - Inventory

Class: Payment Processor	
Responsibilities - Validate and Charge Payments - Handle Refunds	Collaborators - Customer - Order

Class: InvoiceService	
Responsibilities - Create Invoice - Generate PDF	Collaborators - Order

Class: EmailService	
Responsibilities - Send Confirmation - Send Updates	Collaborators - Customer - Order

The Smell Test, and the Cures

✗ SMELLS (what you see on the card)

- ✗ Responsibilities fill the card and wrap to the back
- ✗ A responsibility explains HOW (“…using AES-256…”)
- ✗ One responsibility hides an “and” (“validate and email and log…”)
- ✗ The same responsibility lives on two different cards
- ✗ Collaborators outnumber responsibilities
- ✗ A card never gets called during role-play

✓ CURES (what you do about it)

- ✓ Split the class by concern (slide 23 rehab)
- ✓ Rewrite WHAT-style; defer HOW to code
- ✓ Cut the “and”, each half becomes its own line (or card)
- ✓ Move the responsibility to its rightful owner
- ✓ Ask why each collaborator is there; delete the tourists
- ✓ Delete the dead card, or give it a real job

◆ **Iterative refinement:**

CRC is a conversation, not documentation. Beck & Cunningham designed it to be cheap to draw, argue with, and redraw, refine early, refine often; moving a responsibility between cards costs ten seconds and saves ten days.

Six Things to Carry Out of This Room

1. Class = blueprint at design time; object = instance at run time with type, identity, state, behavior.
2. Grammar generates candidates; judgment decides, filter with: state? behavior? many instances?
3. Classes play roles: data manager · source/sink · view · helper. Dependencies point view → data, never the reverse.
4. Not every object is real: the domain gives nouns, the requirements give managers.
5. CRC = Class / Responsibilities / Collaborators. Responsibilities are short WHAT-style verb phrases.
6. High cohesion + low coupling. Role-play finds the gaps. Cards are disposable, iterate.

If you remember one sentence: **“Design the responsibilities, and the classes design themselves.”**